ЛАБОРАТОРНА РОБОТА №10

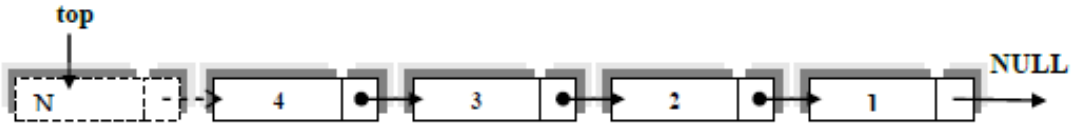
Стеки та черги

Мета: набуття практичних вмінь та навичок по роботі з стеками та чергами.

7.1 Теоретичні відомості

Стек (stack) – лінійний список, у якому всі видалення й доповнення (і звичайно всякий доступ) робляться з одного кінця списку.

У стеці елемент додаються й віддаляються тільки з одного кінця. На малюнку це елемент N. Тобто якщо він додався, то видалятися може спочатку тільки він, а вже потім всі інші.



Для стеку характерні такі властивості:

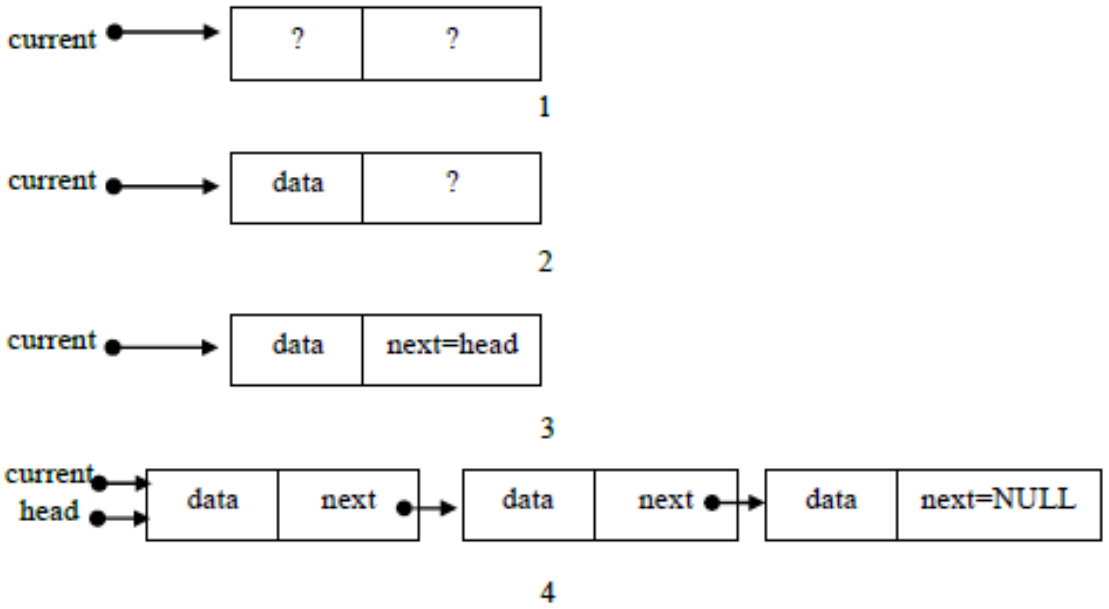
–елементи додаються у вершину (голову) стеку;

–елементи видаляються з вершини (голови) стеку;

–покажчик в останньому елементі стеку дорівнює NULL;

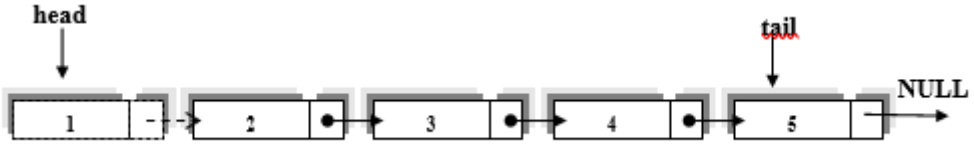
–неможливо вилучити елементи стеку, не вилучивши всі елементи, що йдуть попереду.

Графічне представлення алгоритму вставки елемента до стеку



Черга (queue) – лінійний список, у якому всі видалення відбуваються на одному кінці списку, а всі включення (і звичайно всякий доступ) робляться на іншому його кінці.

Інакше кажучи, у черги є голова (head) і хвіст (tail). Елемент, що додається в чергу, виявляється в її хвості.



У черзі новий елемент додається тільки з одного кінця. Видалення елемента відбувається на іншому кінці. Черга, це по суті однонаправлений список, тільки додавання й видалення елементів відбувається на кінцях списку.

Черга характеризується такими властивостями:

–елементи додаються в кінець черги;

–елементи зчитуються та видаляються з початку (вершини) черги;

–покажчик в останньому елементі черги дорівнює NULL;

–неможливо отримати елемент із середини черги, не вилучивши все елементи що ідуть попереду.

Приклад організації стеків

```
struct cellOfStack {
char liter;
cellOfStack *next;
};
class Stack {
cellOfStack *top; // вказівник на вершину стеку
public:
Stack(){top=NULL;}
void add(char x); // Push
void del(); // Pop;
char Top();
void print();
};
void Stack::add(char x){
cellOfStack *p;
p= new cellOfStack;
p->liter=x;
if (top==NULL){top=p;top->next=NULL;} else {p->next=top;top=p;}
}
int main()
{
Stack S;
for (int i=0;i<=10;i++)//дайте собі відповідь на питання: що
робить цей цикл?
{     S.add(10*i);}
S.print();//переконайтеся у правильності своєї відповіді
S.del(); // Ooooooops!Щось у програмі бракує
S.print();//що сталося зі стеком?
return 0;
}
```

А ось і черги

```
struct CellOfQueue {
int number;
CellOfQueue *next;
};
class Queue {
public:
CellOfQueue *header;
CellOfQueue *tail;
Queue() {header=NULL; tail=NULL;}
void add (int x); //EnQueue
void print();
int front(); //Front
void del();//DeQueue
};
```

```
void Queue::add (int x) {
CellOfQueue *p;
p=new CellOfQueue;
p->number=x;
p->next=NULL;
cout<<"added"<<p->number<<endl;
if (header==NULL){header=p;tail=p;tail->next=NULL;}
else if (header==tail){header->next=p;tail=p;}
else{
tail->next=p;
tail=p;
}
}
void Queue::del() { header=header->next; }
```

7.2 Завдання

Реалізувати аналогічні програми організації стеку та черги для роботи зі своїм варіантом структури.

7.3 Контрольні запитання

При підготовці до лабораторної роботи необхідно опрацювати такі питання:

1. Що таке стек?
2. Які основні функції по роботі зі стеком?
3. Чи потрібно стек організовувати на основі двозв’язних списків?
4. Що таке черга?
5. Які основні функції по роботі з чергою?
6. Чим відрізняються черги від простих списків?
7. Чим відрізняються стек і черга?
8. Які структурні елементи обов’язково містить стек?
9. Які структурні елементи обов’язково містить черга?
10. На основі яких списків доцільно організовувати черги?
11. Наведіть порівняльну характеристику стеків і звичайних однозв’язних списків.
12. Наведіть порівняльну характеристику черг і одно- та двозв’язних списків.